

Timing in this project — and what WNS actually means

A from-zero guide to setup, hold, slack and timing closure, built around the measured results of the 4×4 weight-stationary systolic-array multiplier on a Digilent ZedBoard

Target part `xc7z020c1g484-1` · Constraint `create_clock -period 10.000` (100 MHz) · Design under test `accelerator_system_top` (unmodified)

Result: **positive WNS on every reported build** · **17 clock cycles (170 ns) per 4×4 matrix multiplication**, 13 (130 ns) reusing weights · **0.145 W total on-chip power**

Implementation metrics from the routed-design reports in `FPGA_IMPL/reports/` (Vivado 2020.1, run of 4 Aug 2026) · Companion documents: `implementation_method.pdf`, `VIO_ILA.pdf`, `USB_UART.pdf`

Contents

1. Read this first — the one-page summary
2. Why timing analysis exists at all
3. The anatomy of a timing path
4. The setup equation — where WNS comes from
5. The hold equation — the other half
6. Slack, WNS, TNS and friends — every term defined
7. Endpoints, path groups and what "0 failing" means
8. Why simulation passing is not timing passing
9. This design's clock and constraint
10. Where the timing paths in this design actually are
11. The critical path — the PE multiply-accumulate
12. **Measured results — the full implementation metric set**
13. Reading the numbers: margin, $F_{\max}$, headroom
14. **Cycles per matrix multiplication — latency and throughput**
15. Why this design closes timing so comfortably
16. What erodes the margin — debug cores and routing
17. How to read a Vivado timing report line by line
18. What to do when WNS is negative — ordered checklist
19. Hold violations — a different animal
20. Clock domain crossing and this design
21. The constraint file, annotated
22. Reproducing every number in this document
23. **WNS for preparation — interview / viva question bank**
24. Glossary and cheat sheet

WHO THIS DOCUMENT IS FOR

Someone who has written RTL and run a simulation, but has never had to answer "does it meet timing?". Sections 2–8 assume nothing at all. Sections 9–17 apply that theory to this specific accelerator, using figures actually measured on `xc7z020c1g484-1` and taken from the routed-design reports in `FPGA_IMPL/reports/`. Section 23 is a question bank for interviews and vivas, where "what is WNS?" is one of the most commonly asked questions in digital design and FPGA roles.

1. Read this first — the one-page summary

An FPGA design is a sea of flip-flops separated by combinational logic. Every clock edge, each flip-flop launches a value that has to arrive at the next flip-flop, settle, and be stable a little before the *next* edge captures it. **Static Timing Analysis (STA)** is the tool that checks, for every such path, whether the value gets there in time. It does this exhaustively and without simulating anything — no test vectors, no waveforms, just delay arithmetic over the whole netlist.

Slack is how much time to spare a path has. **WNS — Worst Negative Slack — is the slack of the single worst setup path in the whole design.** It is one number that answers "does the design run at the requested clock frequency?":

WNS > 0 → every path arrives in time. **The design meets timing.**
WNS = 0 → the worst path arrives exactly on the deadline. Passing, zero margin.
WNS < 0 → at least one path is too slow. **The design fails timing** at this clock and may produce wrong results in hardware.

Despite the name, WNS is reported even when it is positive — Vivado prints "Worst Negative Slack" and then shows +0.714 ns. It is really "worst slack"; the word "negative" is historical. A positive WNS is the thing you want, and it is what this project achieved.

What we achieved

Build	WNS	WHS	WPWS	Verdict
Accelerator alone, out-of-context synthesis (no debug logic)	+1.279 ns	+0.263 ns	+3.750 ns	0 failing endpoints of 1 305. ~12.8 % margin on a 10 ns period.
ZedBoard debug build — vio_debug_top with VIO + ILA + debug hub	+0.714 ns	+0.045 ns	+3.750 ns	0 failing endpoints of 5 181. "All user specified timing constraints are met."

Both builds are **positive on setup, positive on hold, and positive on pulse width, with zero failing endpoints and zero total negative slack**. That is a clean pass. Section 12 gives the full report text and every other implementation metric; section 13 explains exactly how much headroom those numbers represent.

The headline numbers, all in one place

Everything below is measured, not estimated, and every figure is traceable to a named report in `FPGA_IMPL/reports/` or to the RTL itself. Section 12 shows where each one comes from.

Category	Metric	Measured value	Source
Timing	WNS / TNS / failing endpoints (setup)	+0.714 ns / 0.000 / 0 of 5 181	Design Timing Summary, post-route
	WHS / THS / failing endpoints (hold)	+0.045 ns / 0.000 / 0 of 5 181	Design Timing Summary
	WPWS / TPWS / failing endpoints	+3.750 ns / 0.000 / 0 of 2 950	Design Timing Summary
	Worst bus-skew slack (CDC)	+9.253 ns of a 10.000 ns requirement	*_bus_skew_routed.rpt
Frequency	Constrained clock	100.0 MHz (10.000 ns)	vio_debug.xdc
	Setup margin on the period	7.1 %	derived from WNS
	Estimated F _{max}	≈ 107.7 MHz	derived, see §13
Performance	Cycles per 4×4 matrix multiplication	17 cycles 13 reusing weights	controller.sv , see §14
	Wall-clock latency of one 4×4 multiply	170 ns at 100 MHz 130 ns reusing weights	17 (or 13) × 10 ns
	Peak arithmetic rate (COMPUTE phase)	1.6 GMAC/s = 3.2 GOP/s	16 PEs × 100 MHz
	Average rate over one 4×4 job	0.38 GMAC/s (23.5 % of peak)	64 MACs / 17 cycles
Resources (whole debug build, post-route)	LUTs as logic / as memory	4 079 (7.67 %) / 442	*_power_routed.rpt §1.1
	Registers	6 370 (5.99 %)	*_power_routed.rpt §1.1
	Block RAM tiles	11 of 140 (7.86 %)	*_power_routed.rpt §1.1
	DSP48E1 slices	0 of 220 (0.00 %)	*_utilization_synth.rpt
Power	Total on-chip power	0.145 W	*_power_routed.rpt
	Dynamic / static split	0.042 W / 0.104 W	*_power_routed.rpt
	Junction temperature / max ambient	26.7 °C / 83.3 °C	*_power_routed.rpt
Sign-off	DRC errors / critical warnings	0 / 0 (2 warnings)	*_drc_routed.rpt
	Methodology TIMING-* violations	0 (16 warnings, none timing)	*_methodology_drc_routed.rpt

2. Why timing analysis exists at all

A flip-flop is not instantaneous. It has three physical requirements, and every one of them is a promise the silicon vendor makes and the designer must not break:

Parameter	Symbol	What it means physically
Clock-to-Q delay	T_{co} / T_{cq}	After a clock edge arrives, the flip-flop's output does not change immediately. It takes a small propagation delay before the new value appears on Q.
Setup time	T_{su}	The data at the D input must be stable for this long before the capturing clock edge. Arrive later than that and the flip-flop may not resolve the value correctly.
Hold time	T_h	The data at D must remain stable for this long after the clock edge. Change too soon and the flip-flop can capture the <i>new</i> value instead of the old one.

Violating setup or hold does not produce a neat error. It produces **metastability**: the flip-flop output hovers at an undefined level for an unpredictable time before settling to either 0 or 1, effectively at random. Downstream logic sees garbage. The failure is data-dependent, temperature- dependent, and chip-to-chip variable — which is exactly why it must be proven *analytically* before the bitstream is built, not hunted for in the lab.

THE CENTRAL POINT A BEGINNER MUST INTERNALISE

Simulation checks

WHAT

your logic computes. Timing analysis checks

WHETHER THE SILICON CAN COMPUTE IT FAST ENOUGH

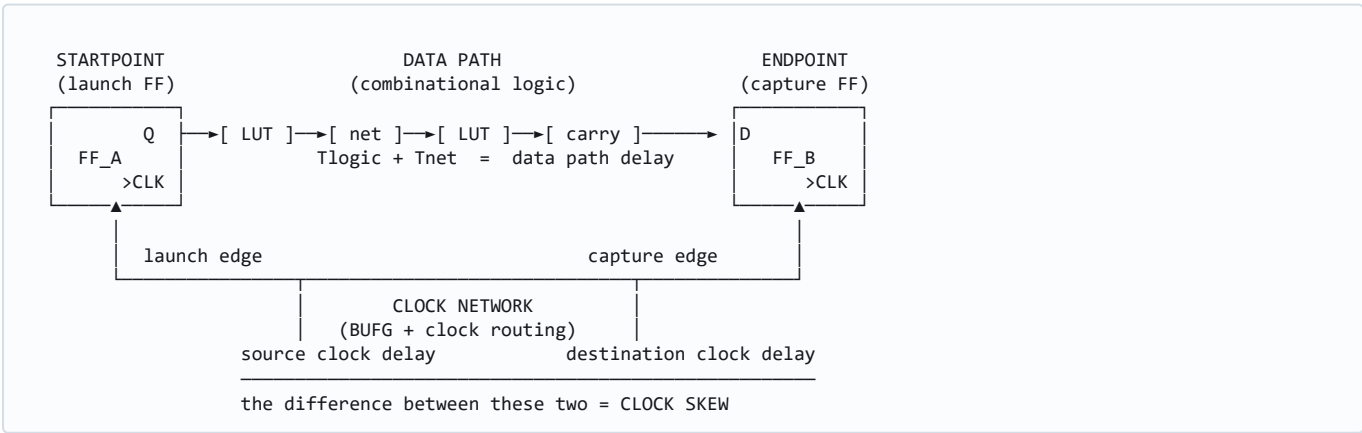
. They are completely independent. A design can pass 3 680 simulation checks with 0 mismatches — as this one does — and still fail in hardware if WNS is negative. Both must pass.

Static, not dynamic

STA is called *static* because it never applies stimulus. It walks the netlist graph, sums up cell and wire delays along every register-to-register path, and compares the total against the clock period. That makes it **exhaustive** — it cannot miss a path the way a simulation can miss a corner case — at the price of being **pessimistic** (it will happily analyse paths that can never be exercised in real operation, which is what timing exceptions in section 18 are for).

3. The anatomy of a timing path

Every path STA analyses has exactly four parts. Learn these four words and the rest follows.



Term	Definition
Startpoint	Where the data is launched: a flip-flop clock pin, or an input port.
Endpoint	Where the data is captured: a flip-flop D pin, or an output port. Vivado's "number of endpoints" is a count of these capture points — 5 181 of them in our debug build.
Data path delay	Clock-to-Q of the launch flop, plus every LUT / carry / MUX delay, plus every wire (net) delay in between. In modern FPGAs the net delay is often <i>more than half</i> the total.
Clock path delay	How long the clock edge itself takes to reach each flip-flop, through the global buffer (BUFG) and clock routing.
Clock skew	Destination clock delay minus source clock delay. Positive skew (clock arrives at the capture flop <i>later</i>) helps setup and hurts hold. Negative skew does the opposite.
Clock uncertainty	A safety margin the tool subtracts for jitter, PLL/MMCM phase error, and modelling imprecision. You get it for free; it always makes the analysis stricter.
Levels of logic	How many LUTs the signal passes through on the path. A high number is the classic cause of a long path.

4. The setup equation — where WNS comes from

Setup asks: **does the data arrive early enough for the next edge?** The launch edge and the capture edge are one clock period apart (for a normal single-cycle path).

$$\text{Data Arrival Time} = T_{\text{launch-clk}} + T_{\text{co}} + T_{\text{logic}} + T_{\text{net}}$$

$$\text{Data Required Time} = T_{\text{capture-clk}} + T_{\text{period}} - T_{\text{setup}} - T_{\text{uncertainty}}$$

$$\text{Setup Slack} = \text{Data Required Time} - \text{Data Arrival Time}$$

Expand it and the intuition is plain:

$$\text{Setup Slack} = T_{\text{period}} - (T_{\text{co}} + T_{\text{logic}} + T_{\text{net}} + T_{\text{setup}}) + T_{\text{skew}} - T_{\text{uncertainty}}$$

The clock period is your budget. Clock-to-Q, the logic, the wires and the setup requirement are the spending. What is left over is slack. Note what this tells you immediately:

- **Raise the frequency** (shrink T_{period}) and slack falls. This is why the same netlist can pass at 100 MHz and fail at 200 MHz — nothing about the logic changed, only the budget.
- **Add logic levels between flops** and slack falls. Deep combinational clouds are the number one cause of setup failure.
- **Pipelining fixes setup** — inserting a register splits one long path into two short ones, each getting a full period.

The worst setup slack across all paths is WNS. That is the whole definition.

A worked number

Suppose $T_{\text{period}} = 10.000$ ns, $T_{\text{co}} = 0.400$ ns, logic + net = 8.500 ns, $T_{\text{setup}} = 0.100$ ns, skew = +0.050 ns, uncertainty = 0.035 ns:

$$\text{Slack} = 10.000 - (0.400 + 8.500 + 0.100) + 0.050 - 0.035 = \mathbf{+1.015 \text{ ns}} \rightarrow \text{passes, } \sim 10 \% \text{ margin}$$

Now ask for 150 MHz instead — $T_{\text{period}} = 6.667$ ns — and with the identical logic:

$$\text{Slack} = 6.667 - 9.000 + 0.050 - 0.035 = \mathbf{-2.318 \text{ ns}} \rightarrow \text{fails}$$

5. The hold equation — the other half

Hold asks the opposite question: **does the data arrive too early?** If a path is so short that the new value races through the logic and reaches the capture flop *before that same clock edge has finished capturing the old value*, the flip-flop captures the wrong data. The launch and capture edges here are the **same** edge — there is no period term.

$$\begin{aligned}\text{Hold Slack} &= \text{Data Arrival Time} - (T_{\text{capture-clk}} + T_{\text{hold}}) \\ &= (T_{\text{co}} + T_{\text{logic}} + T_{\text{net}}) - T_{\text{hold}} - T_{\text{skew}}\end{aligned}$$

THE SINGLE MOST IMPORTANT ASYMMETRY IN ALL OF STA

The clock period does not appear in the hold equation. Therefore

SLOWING THE CLOCK DOWN CANNOT FIX A HOLD VIOLATION

. A design with negative WHS is broken at every frequency, including 1 Hz. Setup failures are a speed problem; hold failures are a structural problem fixed by adding delay (buffers, routing detours) or fixing skew. On FPGAs the placer and router fix hold automatically in almost all cases — a leftover hold violation usually means a genuine design error such as an unconstrained clock crossing.

The worst hold slack across all paths is **WHS — Worst Hold Slack**. Our debug build reports **WHS = +0.045 ns**: tight in absolute terms, but positive, with 0 failing endpoints. Small positive hold slack is entirely normal and not a concern — the router targets "just enough" hold margin because over-fixing hold costs setup margin.

6. Slack, WNS, TNS and friends — every term defined

Acronym	Full name	Meaning, and how to read it
WNS	Worst Negative Slack	The smallest setup slack of any path in the design . One number. Positive → all setup paths pass. This is the headline timing metric and the answer to "did it close?".
TNS	Total Negative Slack	The sum of the slacks of all failing setup endpoints (passing endpoints contribute 0). TNS = 0 means nothing fails. TNS tells you <i>how much</i> of the design is failing, where WNS tells you only <i>how badly the worst one</i> fails. WNS = -0.05 ns with TNS = -0.05 ns is one lazy path; WNS = -0.05 ns with TNS = -900 ns is a design-wide problem.
WHS	Worst Hold Slack	The smallest hold slack of any path. Must be positive. Cannot be fixed by lowering frequency.
THS	Total Hold Slack	Sum over all failing hold endpoints. Should be 0.000.
WPWS	Worst Pulse Width Slack	Checks that the clock <i>pulse itself</i> is legal — that its high time and low time are longer than the minimum the flip-flop, BRAM or DSP requires, and that the clock period is not shorter than the cell's own minimum period. This catches broken clock division, badly configured MMCMs, and clocks that are too fast for a hard block.
TPWS	Total Pulse Width Negative Slack	Sum over failing pulse-width checks. Should be 0.000.
Failing endpoints	—	How many capture points have negative slack. 0 is the only acceptable value for sign-off.
Total endpoints	—	How many capture points were analysed at all. Useful as a sanity check: if this number is suspiciously small, large parts of your design are unconstrained and therefore not being checked .
F _{max}	Maximum frequency	Not printed directly. Derive it: $F_{\text{max}} = 1 / (T_{\text{period}} - \text{WNS})$. It is an estimate, valid near the constrained frequency only.

THE THREE LINES THAT CONSTITUTE A PASS

A Vivado Design Timing Summary is a pass if and only if all three of these hold, and the report says "*All user specified timing constraints are met.*":

Setup:	WNS > 0	TNS = 0.000	Failing Endpoints = 0
Hold:	WHS > 0	THS = 0.000	Failing Endpoints = 0
Pulse Width:	WPWS > 0	TPWS = 0.000	Failing Endpoints = 0

7. Endpoints, path groups and what "0 failing" means

Vivado groups paths so you can see *which kind* of path is the problem. In the timing summary GUI these appear as a tree:

Group	What lives there
Intra-Clock Paths	Launch and capture flops on the same clock. In this project, essentially the entire design: everything runs on <code>sys_clk</code> .
Inter-Clock Paths	Launch and capture on different clocks. Real CDC lives here. In this project, only the JTAG/debug-hub boundary — and Vivado's debug IP constrains that itself.
Other Path Groups	Special groups such as <code>**async_default**</code> — paths through asynchronous set/reset pins.
User Ignored Paths	Paths you removed from analysis with <code>set_false_path / set_clock_groups</code> . Always read this list. A design "passes" trivially if you have falsely ignored the failing paths.
Unconstrained Paths	Paths with no clock definition reaching them. These are not analysed at all. They are invisible in WNS and are the classic way a broken design shows a beautiful timing summary.
Check Timing	A separate audit (our build shows 8 377 entries) listing structural issues: <code>no_clock</code> , <code>constant_clock</code> , <code>unconstrained_internal_endpoints</code> , <code>no_input_delay</code> , <code>no_output_delay</code> , etc. High counts here are normal for a PL-only design with no external I/O interface, but they must be understood, not ignored.

"0 FAILING ENDPOINTS" IS ONLY MEANINGFUL TOGETHER WITH THE ENDPOINT COUNT

0 failing out of 5 181 analysed is a real pass. 0 failing out of 12 analysed means you forgot to write a `create_clock` and the tool checked almost nothing. Always read the two numbers as a pair, and always run `check_timing` and `report_clocks` before believing a summary.

8. Why simulation passing is not timing passing

This project has a very strong functional baseline: **11 of 11 testbenches, 3 680 checks, 0 mismatches** in the QuestaSim re-verification regression. It is worth being precise about what that does and does not prove.

Question	Answered by simulation (RTL)	Answered by STA
Is the maths right — does $C = A \times W$?	Yes. 3 680 checks say so.	No. STA has no idea what the design computes.
Is the FSM phase sequence right (WLOAD 4, COMPUTE N, DRAIN 8, DONE 1)?	Yes.	No.
Will the silicon actually hit 100 MHz?	No. RTL simulation is zero-delay; every gate is instantaneous.	Yes. That is exactly what WNS answers.
Is there a setup violation on a 32-bit adder in a PE?	No.	Yes.
Is the clock pin constrained to the right package pin?	No.	Partly — <code>check_timing</code> flags the missing constraint; the XDC does the placement.

The two are orthogonal and both are mandatory. A design that computes the right answer at a clock frequency it cannot sustain will produce *wrong answers in hardware while simulating perfectly*, which is one of the most confusing failure modes a beginner can meet. That is why the hardware bring-up in `implementation_method.pdf` re-runs the **same golden T5 vector** on the board: it proves the functional result *and* the timing result at once.

9. This design's clock and constraint

The entire accelerator is a **single-clock, fully synchronous design**. There is exactly one clock, and it is constrained in one line:

```
## ---- 100 MHz on-board PL clock (Bank 13, GCLK) -----
set_property -dict {PACKAGE_PIN Y9 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -name sys_clk -period 10.000 [get_ports clk]
```

Property	Value	Consequence for timing
Clock source	ZedBoard 100 MHz PL oscillator, package pin Y9	A real board oscillator, not a PS-derived FCLK. The PL runs even with no PS software loaded.
Period constraint	10.000 ns	The budget every path is measured against. This is T_{period} in section 4.
Clock name	sys_clk	Naming the clock makes every later constraint and report readable.
Duty cycle	50 % (default)	High time = low time = 5.000 ns. This is what the pulse-width check measures against.
Number of clock domains	1 (plus the debug hub's JTAG domain, self-constrained by the IP)	No CDC to constrain, no <code>set_clock_groups</code> , no asynchronous FIFO. A large class of timing problems simply does not exist here.
Reset	Synchronous , active-low <code>rst_n</code> , from a 65 536-cycle power-on counter	Synchronous resets are analysed as ordinary data paths — no **async_default** group headaches, no recovery/removal surprises.
External I/O timing	None to constrain in the VIO/ILA build	The only physical pin is <code>clk</code> . All data enters and leaves over JTAG, inside Vivado's own debug IP, so no <code>set_input_delay</code> / <code>set_output_delay</code> is required.

WHY "CONSTRAIN THE SOURCE, NOT THE BUFFER OUTPUT"

`create_clock` is applied to `[get_ports clk]` — the package pin — not to the output of the BUFG. Constraining the buffer output makes the tool ignore the buffer's own delay and produces optimistic, wrong numbers. Constrain where the clock physically enters the chip and let the tool propagate it forward.

10. Where the timing paths in this design actually are

To reason about timing you must know what the register-to-register paths look like. This accelerator is a three-level hierarchy, and every level is pipelined:

```
accelerator_system_top (L3)          ← host ports, memories, status
├── weight_mem  4 × 32  (LUTRAM, distributed)
├── a_mem       256 × 32 (BRAM, 1-cycle read latency)
├── output_mem  N × 128 (BRAM)
├── controller  ← 5-state FSM: IDLE/WLOAD/COMPUTE/DRAIN/DONE
├── agu         ← address generator + 8-deep write-strobe pipe
├── wrapper_array_buf (L1)
│   ├── input_buf  ← diagonal skew of activations
│   ├── array      ← 4 × 4 = 16 processing_element instances  ← THE ARITHMETIC
│   └── output_skewbuf ← de-skew of results (delay 3-c on column c)
```

Latency chain (fixed, no back-pressure anywhere):

down a PE column	ROWS-1 = 3	
east to column c	+c	→ 3+c
de-skew pipe	+(3-c)	→ 6, independent of c
a_mem read latency	+1	→ 7 = RESULT_LATENCY
write strobe	+1	→ 8 = STORE_LATENCY
total job, load W	N + 13 clocks	(= 17 clocks = 170 ns for N = 4)
total job, reuse W	N + 9 clocks	(= 13 clocks = 130 ns for N = 4)

Grouped by kind, the timing paths are:

Path class	Typical depth	Comment
PE psum accumulation — <code>psum_q</code> → next row's <code>psum_q</code>	1 mux + 8×8 multiply + 32-bit add + 1 mux	The critical path. See section 11.
PE activation march — <code>a_q</code> → east neighbour's <code>a_q</code>	1 net, 0 logic levels	Trivially fast: a straight register-to-register hop.
BRAM read — <code>a_mem</code> / <code>output_mem</code> output → consumer flop	Hard block clock-to-out + routing	BRAM T_{CO} is significant on 7-series but the consumer logic here is shallow.
LUTRAM read — <code>weight_mem</code> (4 × 32 distributed RAM)	LUT delay + routing	Only 40 LUTs as memory; a deeper weight memory would be dead silicon <i>and</i> slower.
FSM state — <code>controller</code> state register feedback	A few LUT levels	6 states, tiny next-state function. Never critical.
AGU counters and write-strobe pipe	8-bit increments, 8-deep shift	Counters and shift registers are the fastest structures the fabric offers.
128-bit result fan-out — <code>output_mem</code> → <code>o_rdata</code> → VIO probe	Wide but shallow	Wide buses cost routing resources, not logic depth.

11. The critical path — the PE multiply-accumulate

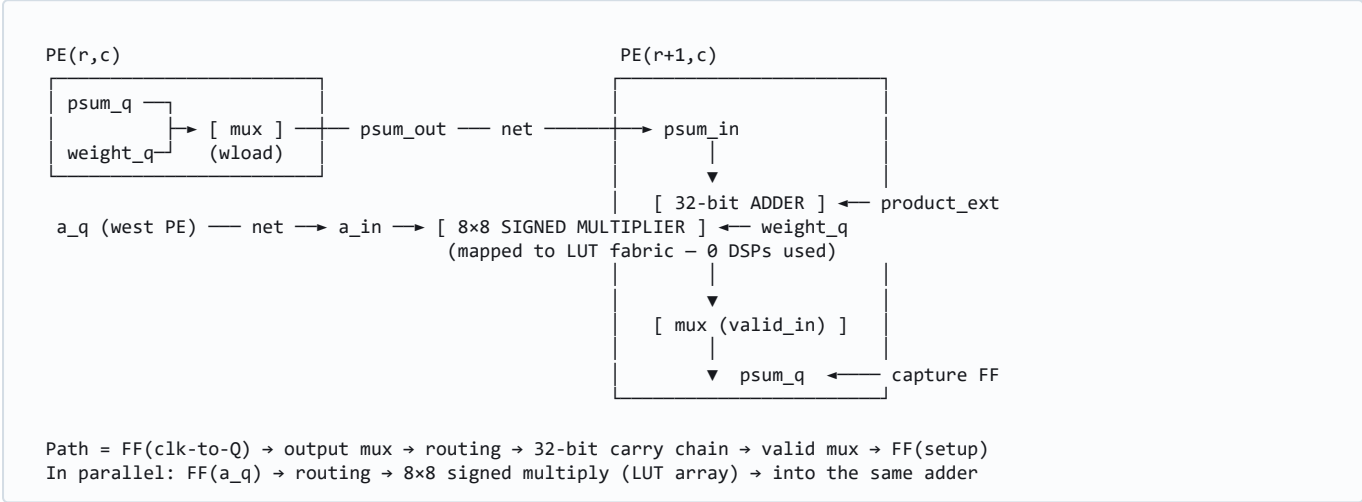
The longest combinational cloud in this design is inside `processing_element` (`MAC.sv`). This is the only module in the whole accelerator that contains arithmetic, and there are sixteen copies of it:

```
// Combinational arithmetic
assign product_ext = 32'(weight_q * a_in);    // signed 8 x signed 8, sign-extended to 32
assign adder_out   = psum_in + product_ext;    // 32-bit signed add

always_ff @(posedge clk) begin
    ...
    psum_q <= valid_in ? adder_out : psum_in;  // accumulate OR bypass
end

assign psum_out = wload ? 32'(weight_q) : psum_q;  // COMBINATIONAL output mux
```

Trace the worst path from one row of the array to the next:



Path = FF(clk-to-Q) → output mux → routing → 32-bit carry chain → valid mux → FF(setup)
In parallel: FF(a q) → routing → 8x8 signed multiply (LUT array) → into the same adder

Why 0 DSPs is the interesting part

The measured utilisation report says **DSPs: 0 of 220 (0.00 %)**. Vivado mapped all sixteen 8×8 signed multipliers into **general LUT fabric**, not into DSP48E1 slices. The consequences:

Consequence	Detail
Slower per multiply	A fabric multiplier is a tree of LUTs and carry logic. A DSP48E1 has a hardened multiplier and is faster and far more compact.
Explains the LUT count	1 564 LUTs for a design this small is dominated by sixteen fabric multipliers plus sixteen 32-bit adders.
It is still fast enough	An 8×8 multiply is small. The synthesiser judged the fabric implementation would meet 10 ns, and the measurement (+1.279 ns / +0.714 ns) confirms it.
It is the first lever if you ever raise the clock	Forcing DSP inference — (<code>* use_dsp = "yes" *</code>) on the module, or <code>synth_design -max_dsp</code> tuning — would shorten this path substantially and free ~1 000 LUTs. It is the single highest-value change if you ever target 200 MHz.

WHY THE ADDER, NOT THE MULTIPLIER, IS USUALLY THE DEEPER HALF

An 8×8 product needs only a shallow partial-product tree. The

32-BIT ADDITION

that follows it propagates a carry across 32 bits — on 7-series that maps to the dedicated CARRY4 chain, which is fast but is a genuine serial dependency. Add the two output muxes and you have the design's longest cloud. This is also why the accumulator width matters: widening `psum` to 48 bits would directly cost setup slack.

An 8×8 product needs only a shallow partial-product tree. The

32-BIT ADDITION

that follows it propagates a carry across 32 bits — on 7-series that maps to the dedicated CARRY4 chain, which is fast but is a genuine serial dependency. Add the two output muxes and you have the design's longest cloud. This is also why the accumulator width matters: widening `psum` to 48 bits would directly cost setup slack.

12. Measured results — the full implementation metric set

Everything in this section is a measurement, not a projection. Sub-sections 12.1 and 12.2 are the timing summaries; 12.3 onward covers the rest of the reports Vivado produced for the routed design, which live in `FPGA_IMPL/reports/`. Signing off a build means reading *all* of these, not only the WNS line.

Report file (<code>FPGA_IMPL/reports/</code>)	Design state	What it answers
<code>vio_debug_top_utilization_synth.rpt</code>	Synthesized	How much of the device the logic occupies, by primitive.
<code>vio_debug_top_control_sets_placed.rpt</code>	Placed	How well the flip-flops pack into slices — a QoR and routability indicator.
<code>vio_debug_top_io_placed.rpt</code>	Placed	Which package pins are used, at which I/O standard, in which bank.
<code>vio_debug_top_clock_utilization_routed.rpt</code>	Routed	Which clock buffers exist, how many loads each drives, which clock regions they span.
<code>vio_debug_top_bus_skew_routed.rpt</code>	Routed	Whether multi-bit CDC buses arrive together enough to be safe.
<code>vio_debug_top_power_routed.rpt</code>	Routed	On-chip power, thermal margin, and where the power goes by hierarchy.
<code>vio_debug_top_drc_opted.rpt</code> / <code>_drc_routed.rpt</code>	Synth / Routed	Physical design-rule violations. Errors here block the bitstream.
<code>vio_debug_top_methodology_drc_routed.rpt</code>	Routed	Xilinx methodology rules — including the TIMING-* family.

12.1 Build A — accelerator alone, out-of-context synthesis

Out-of-context synthesis of `accelerator_system_top` with no VIO, no ILA and no debug hub, on `xc7z020c1g484-1` in Vivado 2020.1, constrained at 100 MHz. Synthesis completed with **0 errors, 0 warnings, 0 critical warnings**.

```
Constraint : create_clock -period 10.000 (100 MHz)
```

```
WNS  = +1.279 ns      0 failing endpoints of 1305
WHS  = +0.263 ns      0 failing endpoints of 1305
WPWS = +3.750 ns      0 failing endpoints of 838
```

Resource	Used	Available	%	Note
Slice LUTs	1 564	53 200	2.94	1 524 as logic + 40 as memory
Slice Registers	792	106 400	0.74	Pipeline flops dominate
Block RAM Tile	2.5	140	1.79	<code>output_mem</code> (32 kbit) + <code>a_mem</code> (8 kbit)
DSPs	0	220	0.00	All sixteen 8×8 multipliers in LUT fabric — see §11
LUT as Memory	40	17 400	0.23	<code>weight_mem</code> 4 × 32 distributed RAM

12.2 Build B — the ZedBoard debug build (VIO + ILA + debug hub)

This is the build you actually program onto the board for Method 2. It is `vio_debug_top`: the same unmodified accelerator, plus a VIO core (6 input probes, 11 output probes), an ILA core (10 probes), three `oneshot` pulse generators, the sticky-done logic, and the auto-inserted `dbg_hub`. This is the **post-route** Design Timing Summary (Vivado's `timing_1` report on the implemented design) as captured in `synthesis_screenshots/` — these are sign-off numbers, not estimates. Every report in `FPGA_IMPL/reports/` comes from the same implementation run:

	Setup	Hold	Pulse Width
Worst slack	WNS 0.714 ns	WHS 0.045 ns	WPWS 3.750 ns
Total slack	TNS 0.000 ns	THS 0.000 ns	TPWS 0.000 ns
Failing endpoints	0	0	0
Total endpoints	5 181	5 181	2 950

“All user specified timing constraints are met.”

Vivado prints that sentence only when every setup, hold and pulse-width check passes. It is the formal statement that the design has **closed timing**. The Clock Summary shows **1** clock — confirming the single-domain structure described in section 9.

READING THE ENDPOINT COUNT AS A CROSS-CHECK

Build A analysed 1 305 endpoints; Build B analyses

5 181

— roughly four times as many. That jump is the debug infrastructure: the ILA's capture registers, the VIO's probe registers, and the debug hub's JTAG shift chains are all real flip-flops that must meet timing. The count going *up* when you add debug cores is exactly what you should expect, and seeing it is a cheap confirmation that the debug logic really is in the netlist and was not optimised away.

12.3 Resource utilisation — two different numbers, and why

The utilisation report is taken after synthesis, when `vio_0` and `ila_0` are still **black boxes**: their internal cells are not counted. The power report is taken after routing, when everything is real. Both are true; they answer different questions, and quoting one where the other is meant is a common way to mis-state a design's size.

Post-synthesis — `vio_debug_top_utilization_synth.rpt`

Site type	Used	Available	Util %	Note
Slice LUTs	1 889	53 200	3.55	1 848 as logic + 41 as memory
LUT as Distributed RAM	32	17 400	—	<code>weight_mem</code> 4 × 32 (<code>RAMS32</code> × 32)
LUT as Shift Register	9	17 400	—	The AGU write-strobe pipe, inferred as <code>SRL16E</code>
Slice Registers	1 098	106 400	1.03	All flip-flops; 0 latches — no accidental latch inference anywhere
F7 / F8 Muxes	0	26 600 / 13 300	0.00	No wide multiplexers — consistent with a shallow control path
Block RAM Tile	2.5	140	1.79	2 × <code>RAMB36E1</code> (<code>output_mem</code>) + 1 × <code>RAMB18E1</code> (<code>a_mem</code>)
DSPs	0	220	0.00	All sixteen 8×8 multipliers in LUT fabric — see §11
Bonded IOB	1	200	0.50	Only <code>c1k</code> . Everything else is JTAG
BUFGCTRL	1	32	3.13	Becomes 2 after the debug hub adds its TCK buffer
MMCM / PLL	0 / 0	4 / 4	0.00	No clock generation at all — the board oscillator is used directly

Primitive census — what the design is actually built from

Primitive	Count	What it is, and where it comes from
FDRE	1 097	D flip-flop with clock enable and synchronous reset — the design's standard register.
FDSE	1	One synchronous-set flop; the sticky-done latch in <code>vio_debug_top</code> .
LUT3 / LUT2 / LUT4 / LUT6 / LUT5 / LUT1	1 006 / 586 / 483 / 439 / 290 / 6	The 2 810 LUTs. The bias toward small LUTs (LUT2/LUT3 dominate) is the signature of the sixteen 8×8 partial-product trees.
CARRY4	348	The dedicated carry chains: sixteen 32-bit accumulators need 8 CARRY4 each = 128, plus the multiplier trees and the AGU counters.
RAMS32	32	<code>weight_mem</code> as distributed RAM.
SRL16E	9	The AGU's 8-deep write-strobe / write-address pipeline.
RAMB36E1 / RAMB18E1	2 / 1	<code>output_mem</code> (N × 128) and <code>a_mem</code> (256 × 32).
IBUF / BUFG	1 / 1	The clock input buffer and the global clock buffer it drives.
<code>vio_0</code> / <code>ila_0</code>	1 / 1	Black boxes at this stage — their thousands of cells are not in the counts above.

Post-route, whole design — `vio_debug_top_power_routed.rpt` §1.1

Once the debug IP is elaborated and routed, the real footprint is roughly **2.2× the LUTs** and **5.8× the registers** of the synthesis figure. That entire difference is debug infrastructure.

Resource	Post-synth	Post-route	Util %	Growth is
LUT as Logic	1 848	4 079	7.67	ILA trigger/compare logic, VIO, debug hub
Registers	1 098	6 370	5.99	ILA capture pipeline + VIO probe registers + JTAG shift chains
CARRY4	348	423	3.18	Mostly already ours — the accelerator's adders
LUT as Distributed RAM	32	56	0.32	Debug-hub FIFOs
LUT as Shift Register	9	386	2.22	ILA probe pipelining is heavily SRL-based
F7 / F8 Muxes	0	191	0.36	Entirely debug IP
Block RAM	2.5	11	7.86	≈ 8 BRAM36 of ILA capture storage — see §16
I/O	1	1	0.50	Unchanged: still just c1k

THE ACCELERATOR IS A SMALL TENANT IN ITS OWN BITSTREAM

Of the 11 block RAM tiles in the routed design, only 2.5 belong to the accelerator; the rest is ILA capture memory. Of the 6 370 registers, 1 097 are ours. Instrumentation is the majority of this build by every resource measure — which is exactly why §16 treats it as the first thing to shrink if margin is ever needed.

12.4 Clock resources — `vio_debug_top_clock_utilization_routed.rpt`

Clock	Period	Buffer / site	Clock loads	Reach
<code>sys_c1k</code>	10.000 ns	BUFG at BUFGCTRL_X0Y0	6 497	4 clock regions: X0Y1 (2 968 loads), X1Y1 (1 243), X0Y2 (1 285), X1Y2 (758)
<code>...bscan_inst/TCK</code>	33.000 ns	BUFG at BUFGCTRL_X0Y1	465	2 clock regions: X0Y2 (204), X1Y2 (258)

Reading it:

- **Both clocks are on true global buffers.** That is what keeps clock skew in the tens of picoseconds and is a precondition for the small, well-behaved WHS in §12.2. A clock accidentally routed on general fabric shows up here immediately.
- **2 BUFGCTRL of 32, 0 MMCM, 0 PLL, 0 BUFR/BUFIO/BUFH.** The design does not synthesise a clock at all — the ZedBoard's 100 MHz oscillator drives the fabric directly. Nothing can be misconfigured because nothing is configured.
- **`sys_c1k` spans four of the six clock regions** and drives 6 254 slice loads. That spread is a consequence of low utilisation (the placer had no reason to cluster), and it costs nothing here because a BUFG reaches the whole device by design.
- **The 33 ns TCK clock is the JTAG domain.** It exists only inside the debug hub, and Vivado's own IP constrains the crossing — see §12.5 and §20.

The clock pin — vio_debug_top_io_placed.rpt

Pin	Signal	Pin name	Standard / bank	Why it matters
Y9	clk	IO_L12P_T1_MRCC_13	LVC MOS33 / bank 13	MRCC = Multi-Region Clock Capable. A clock <i>must</i> enter on an MRCC or SRCC pin to reach a BUFG cleanly; entering on an ordinary I/O pin forces a fabric detour and inflates skew. Placement is FIXED , i.e. it came from the XDC and not from the placer.

Total user I/O: 1. Every other signal in the design enters and leaves over JTAG.

12.5 Bus skew — the CDC check you did not write

A bus-skew constraint is different from setup and hold. It does not ask "did each bit arrive in time?" but "did all the bits of this bus arrive **close enough together**?" — which is the question that matters for a gray-coded pointer crossing between clock domains, where two bits changing on different clock edges corrupts the pointer even though every individual path meets timing.

You did not write these constraints; Xilinx's debug IP supplied them for its own asynchronous FIFOs. Vivado checked them anyway, and they pass with enormous room:

Id	Bus (all inside <code>dbg_hub</code>)	Requirement	Actual skew	Slack
1	RD-FIFO read pointer, TCK → <code>sys_clk</code>	10.000 ns	0.690 ns	+9.310 ns
2	RD-FIFO write pointer, TCK → <code>sys_clk</code>	10.000 ns	0.727 ns	+9.273 ns
3	WR-FIFO read pointer, TCK → <code>sys_clk</code>	10.000 ns	0.747 ns	+9.253 ns ← worst
4	WR-FIFO write pointer, TCK → <code>sys_clk</code>	10.000 ns	0.682 ns	+9.318 ns

All four are 4-bit gray-coded FIFO pointers crossing from the 33 ns JTAG domain into `sys_clk`, each with 4 endpoints. Worst-case actual skew is **0.747 ns against a 10.000 ns budget — 7.5 % of the allowance used**. The report is generated with `-warn_on_violation`, so a failure would appear as a warning rather than an error; there are none.

WHY THE ACCELERATOR CONTRIBUTES NOTHING HERE

Bus-skew checks only exist where a multi-bit bus crosses clock domains. The accelerator is single-clock, so it has no such bus — every one of the four constraints belongs to the debug hub. If you ever add a second clock domain to this design, this report stops being a formality and becomes something you must read.

12.6 Power — `vio_debug_top_power_routed.rpt`

Vector-less power estimation on the routed design, commercial grade, typical process, 25 °C ambient, 250 LFM airflow, no heat sink, medium board:

Quantity	Value	What it tells you
Total on-chip power	0.145 W	Trivial for a ZedBoard, which supplies several watts. No thermal or supply concern whatsoever.
Dynamic power	0.042 W (29 %)	Switching activity. This is the part your design controls.
Device static power	0.104 W (71 %)	Leakage. It is a property of the xc7z020 die, not of your design — you would pay it for an empty bitstream too. Static dominating is the normal signature of a small, slow-ish design on a large part.
Junction temperature	26.7 °C	1.7 °C above the 25 °C ambient. Effectively cold.
Maximum ambient	83.3 °C	The design would still be within thermal limits in an 83 °C room.
Effective θ_{JA}	11.5 °C/W	Package-to-air thermal resistance assumed by the model.
Confidence level	Medium	Read this before quoting the number. Design state, clock activity, I/O activity and device models are all "High"; <i>internal node activity is "Medium"</i> because less than 25 % of internal nodes had user-specified toggle rates. To upgrade it, feed a post-route simulation SAIF into <code>read_saif</code> .

Where the power goes

On-chip component	Power	Comment
Clocks	0.025 W	60 % of all dynamic power is the clock tree itself — 6 497 clock loads toggling every cycle, whether or not the design is doing anything.
Block RAM	0.014 W	Second largest, and mostly the ILA's capture memory.
Signals	0.002 W	9 900 nets.
Slice logic	0.001 W	13 622 cells. Fabric logic is remarkably cheap at this scale.
I/O	< 0.001 W	One pin.
Static	0.104 W	Leakage.

Dynamic power by hierarchy — the interesting table

Instance	Dynamic power	Share	What it is
vio_debug_top (total)	0.042 W	100 %	Whole design
u_ila	0.019 W	45 %	The ILA — the single largest consumer in the design
u_acc (the accelerator)	0.012 W	29 %	Everything this project actually computes with
u_acc/u_datapath/u_output_mem	0.008 W	19 %	The 128-bit result BRAM — two thirds of the accelerator's power
u_acc/u_datapath/u_fabric	0.003 W	7 %	The entire 4×4 PE array — sixteen MACs for 3 mW
u_vio	0.007 W	17 %	VIO probe registers (0.005 W in, 0.001 W out)
dbg_hub	0.003 W	7 %	JTAG shift chains and FIFOs

THE MOST QUOTABLE NUMBER IN THIS DOCUMENT

THE SIXTEEN PROCESSING ELEMENTS — THE WHOLE ARITHMETIC ENGINE — BURN 3 MW.

The memory that stores their results burns 8 mW, and the debug logic watching them burns 29 mW, nearly 2.5× the accelerator itself. This is the classic accelerator result in miniature: *arithmetic is cheap, moving and observing data is expensive*. §14 converts these figures into energy per matrix multiplication.

Supply currents

Rail	Voltage	Total	Dynamic	Static
Vccint	1.000 V	0.048 A	0.041 A	0.008 A
Vccaux	1.800 V	0.010 A	0.000 A	0.010 A
Vccbram	1.000 V	0.002 A	0.001 A	0.001 A
Vccpint / Vccpaux / Vccp11	1.0 / 1.8 / 1.8 V	0.017 / 0.010 / 0.003 A	0	all static
Vccadc	1.800 V	0.020 A	0.000 A	0.020 A
All Vcco* banks	—	0.000 A	0.000 A	0.000 A

The Vccp* rails are the **Processing System**'s. They draw static current even though this is a PL-only design and no software is loaded — that is the Zynq architecture, not a fault, and it is the same fact behind the zps7-1 DRC warning in §12.8. Every Vcco bank reads 0.000 A because only one pin is used and it is an input.

12.7 Control sets — vio_debug_top_control_sets_placed.rpt

A **control set** is the triple {clock, clock-enable, set/reset}. On 7-series, all eight flip-flops in a slice must share one control set, so a design with many distinct control sets packs badly: flops are scattered across more slices, nets get longer, and timing gets harder. It is a leading indicator of routability that nothing else reports.

Statistic	Value	Interpretation
Total control sets	160	For 6 370 registers — about 40 flops per control set. Healthy.
Minimum possible	158	The tool could only have saved 2. There is essentially nothing to optimise.
Added by synthesis replication	2	Deliberate register duplication to cut fan-out.
Added by physical synthesis	0	The placer never had to replicate to meet timing.
Unused register locations in occupied slices	454	Wasted flop sites from imperfect packing. Negligible against 106 400 available.
Control sets with fan-out ≥ 16	71 of 160	Nearly half are large and pack well; only 29 have fan-out below 4.

Flip-flop style distribution — a direct check on the RTL's reset discipline

Clock enable	Synchronous set/reset	Asynchronous set/reset	Registers	Slices
No	No	No	2 559	562
No	Yes	No	1 970	559
Yes	No	No	925	267
Yes	Yes	No	699	187
No	No	Yes	135	44
Yes	No	Yes	82	18

6 153 of 6 370 registers (96.6 %) use synchronous or no reset. The 217 asynchronous-reset flops are all inside the debug hub's FIFO reset logic — they are also the cells behind the four LUTAR-1 methodology warnings in §12.8. Not one asynchronously-reset flop belongs to the accelerator, which is exactly what §9 claims about the reset scheme and is worth knowing because asynchronous resets create ****async_default**** path groups and recovery/removal checks that this design simply never has to argue about.

12.8 DRC and methodology sign-off

Design rule checks — vio_debug_top_drc_routed.rpt

Rule	Severity	Description	Count	Verdict
RTSTAT-10	Warning	No routable loads	1 (25 nets)	Benign. All 25 nets are inside dbg_hub and u_ila — IP outputs (FIFO overflow, spare ctl_reg bits, unused BSCAN taps) that this configuration does not consume. Nothing of ours is dangling.
ZPS7-1	Warning	PS7 block required	1	Expected and deliberate. This is a PL-only design on a Zynq part; no PS7 is instantiated. Handled by BITSTREAM.GENERAL.UNCONSTRAINEDPINS {Allow} in the XDC (§21). It does not affect timing.

0 errors and 0 critical warnings at both checkpoints — _drc_opted.rpt after synthesis shows only ZPS7-1, and routing added only the benign RTSTAT-10. A single DRC error would have blocked write_bitstream entirely.

Methodology — vio_debug_top_methodology_drc_routed.rpt

Rule	Count	Description	Where it is, and what to do
PDRC-190	11	Suboptimally placed synchronised register chain	All eleven are <code>stat_reg</code> → <code>stat_reg_1</code> synchroniser pairs inside <code>dbg_hub</code> , split across different SLICES instead of sharing one. It slightly raises MTBF exposure in Xilinx's own IP. Not ours, and not actionable without floorplanning the debug hub.
LUTAR-1	4	LUT drives asynchronous reset	All four are the <code>dbg_hub</code> FIFO reset generators — a multi-input LUT feeding a <code>PRE</code> pin, which can glitch. Again inside Xilinx IP. The accelerator has no asynchronous resets at all (see §12.7).
SYNTH-6	1	Timing of a RAM block might be sub-optimal	This one is ours, and it is the only genuinely actionable finding in the whole report set. See the box below.

16 warnings, 0 errors, and — the line that matters — zero TIMING-* violations. The TIMING-* family is the one that flags constraint mistakes: missing clock definitions, unconstrained I/O, clock crossings without exceptions, false paths hiding real failures. None fired, which is independent evidence that the clean Design Timing Summary in §12.2 was earned rather than produced by an incomplete set of constraints.

SYNTH-6 : THE ONE FINDING INSIDE THE ACCELERATOR

The timing for the instance `u_acc/u_datapath/u_a_mem/mem_reg`, implemented as a RAM block, might be sub-optimal as no output register was merged into the block.

A 7-series BRAM has an

OPTIONAL INTERNAL OUTPUT REGISTER

. If the RTL's read register is written so that Vivado can absorb it into the BRAM, the read data comes out of a hardened flip-flop right at the memory; if it cannot, the data leaves the BRAM combinationally and is captured by a fabric flop some distance away — costing both the BRAM's clock-to-out advantage and the routing hop. `a_mem` is in the second category.

WHY IT DOES NOT MATTER TODAY:

the `a_mem` read is not the critical path (§11 shows that is the PE MAC), and the design closes with +0.714 ns anyway.

WHY IT WOULD MATTER TOMORROW:

if you take the DSP route in §18 step 6 and shorten the MAC path, the `a_mem` read becomes a candidate for the new critical path — and this is then the next thing to fix. Note that absorbing the output register adds a cycle of read latency, so `RESULT_LATENCY`, `STORE_LATENCY`, the AGU pipe depth and the controller's DRAIN length must all be updated together (§18 step 4).

12.9 Every measured number, one table

Metric	Value	Report
WNS / TNS / failing / total (setup)	+0.714 ns / 0.000 / 0 / 5181	timing summary
WHS / THS / failing / total (hold)	+0.045 ns / 0.000 / 0 / 5181	timing summary
WPWS / TPWS / failing / total	+3.750 ns / 0.000 / 0 / 2950	timing summary
Worst bus skew slack	+9.253 ns (0.747 of 10.000 ns used)	bus_skew_routed
Clocks / BUFG / MMCM / PLL	2 / 2 / 0 / 0	clock_utilization_routed
sys_clk loads / regions	6 497 / 4	clock_utilization_routed
LUT (logic + memory), post-route	4 079 + 442 = 4 521 (8.5 %)	power_routed §1.1
Registers, post-route	6 370 (5.99 %)	power_routed §1.1
CARRY4 / F7-F8 muxes	423 / 191	power_routed §1.1
Block RAM tiles	11 of 140 (7.86 %)	power_routed §1.1
DSP48E1	0 of 220 (0.00 %)	utilization_synth
Bonded IOB	1 of 200 (0.50 %)	io_placed
Total on-chip power	0.145 W (0.042 dyn + 0.104 static)	power_routed
Accelerator dynamic power	0.012 W (PE array alone: 0.003 W)	power_routed §3.1
Junction temp / max ambient	26.7 °C / 83.3 °C	power_routed
Control sets (actual / minimum)	160 / 158	control_sets_placed
Async-reset registers	217 of 6 370 (3.4 %, all in debug IP)	control_sets_placed
DRC errors / crit. warnings / warnings	0 / 0 / 2	drc_routed
Methodology violations (TIMING-*)	16 warnings, 0 timing	methodology_drc_routed
Cycles per 4x4 matrix multiply	17 (170 ns); 13 (130 ns) reusing weights	controller.sv , §14

13. Reading the numbers: margin, $F_{\max}$, headroom

$$\text{Margin (\%)} = \text{WNS} / T_{\text{period}} \times 100$$

$$F_{\max} \approx 1 / (T_{\text{period}} - \text{WNS})$$

Build	WNS	Margin on 10 ns	Longest path takes	Estimated $F_{\max}$
A — accelerator alone	+1.279 ns	12.8 %	10.000 – 1.279 = 8.721 ns	1 / 8.721 ns $\approx$ 114.7 MHz
B — with VIO + ILA	+0.714 ns	7.1 %	10.000 – 0.714 = 9.286 ns	1 / 9.286 ns $\approx$ 107.7 MHz

How to interpret those figures honestly:

- **Both pass, comfortably enough to ship.** Zero failing endpoints, zero TNS.
- **7.1 % is a working margin, not a luxurious one.** It is entirely adequate for a design that is programmed, run and observed on a bench at a fixed 100 MHz. It is *not* the kind of margin that would survive being asked for 150 MHz.
- **The debug cores cost ~0.565 ns of setup slack** (1.279 $\rightarrow$ 0.714). That is the price of observability, and it is worth stating explicitly: instrumenting a design is not free in timing, because the probes contend for the same routing resources as the logic they observe.
- **WHS fell from +0.263 ns to +0.045 ns.** Still positive, still 0 failing endpoints. Tight hold slack is normal after place-and-route because the router deliberately does not over-insert hold buffers — every one of them would eat setup margin.
- **WPWS is +3.750 ns in both builds**, and it is the same number for a reason: a 10 ns clock at 50 % duty has a 5.000 ns high pulse, and the 7-series minimum high pulse for a BUFG-driven flip-flop is 1.250 ns. 5.000 – 1.250 = 3.750 ns. This value is a property of the *clock*, not of your logic, which is why adding the entire debug infrastructure did not move it by a picosecond.

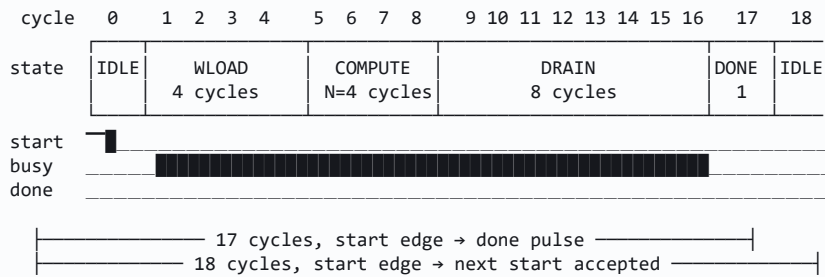
DO NOT TREAT $F_{\max}$ AS A PROMISE

$F_{\max}$ derived from WNS is a linear extrapolation from one operating point. Re-constraining the design at 114 MHz will *not* reliably give WNS = 0 — the tools optimise

TO THE CONSTRAINT

, so a tighter constraint changes placement, routing and even logic mapping. The only way to know the real $F_{\max}$ is to re-run implementation at the new period and read the new WNS.

A. NEW WEIGHT MATRIX - 17 cycles



B. SAME WEIGHTS AS THE PREVIOUS JOB - 13 cycles

cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
state	IDLE	COMPUTE N=4 cycles				DRAIN 8 cycles								DONE 1	IDLE

WLOAD is not entered at all: the array still physically holds the weights.

Useful arithmetic happens only in the 4 COMPUTE cycles.

The other 13 are weight loading (4), pipeline drain (8) and the DONE handshake (1); drop the weight load and only 9 remain.

Period 0 is the reference point, not part of the total - it is what makes the issue interval one larger than the latency.

Question	Answer	Why
Cycles from <code>start</code> to <code>done</code>	17 / 13	The latency of one job, loading / reusing weights. These are the numbers to quote.
Cycles for which <code>busy</code> is high	16 / 12	<code>busy</code> covers WLOAD...DRAIN and is already low when <code>done</code> fires.
Cycles before the next <code>start</code> is accepted	18 / 14	The machine passes through <code>DONE</code> and back to <code>IDLE</code> ; <code>start</code> is only sampled in <code>IDLE</code> . This is the issue interval for back-to-back jobs.
Cycles of actual multiply-accumulate work	4	COMPUTE only — one activation vector per cycle.

A COMMON OFF-BY-ONE, WORTH GETTING RIGHT

`start` is sampled

ONLY IN IDLE

and is ignored while `busy` is high. If a host asserts `start` during a job it is silently dropped — no error, no stall, the command simply never happens. That is the single most common bring-up bug on this design, and it looks exactly like a timing failure from the outside. See [implementation_method.pdf](#) §4 on why the VIO's `start` must go through the `oneshot` module.

14.3 What that means as throughput

One $4 \times 4 \times 4 \times 4$ multiplication is $4 \times 4 \times 4 = \mathbf{64 \text{ multiply-accumulate operations}}$, or 128 arithmetic operations counting the multiply and the add separately.

Measure	Value at 100 MHz	Derivation
Latency, one 4×4 multiply	170 ns	17 cycles $\times$ 10.000 ns
...reusing the loaded weights	130 ns	13 cycles $\times$ 10.000 ns
Back-to-back repeat rate	180 ns $\rightarrow$ 5.56 M multiplies/s 140 ns $\rightarrow$ 7.14 M reusing weights	18- / 14-cycle issue interval
Cycles per output element	1.06 / 0.81	17 or 13 cycles $\div$ 16 results
Peak arithmetic rate (during COMPUTE)	1.6 GMAC/s = 3.2 GOP/s	16 PEs $\times$ 1 MAC/cycle $\times$ 100 MHz. This is the array's ceiling and it is <i>sustained</i> for the whole COMPUTE phase — every PE is busy every cycle.
Average rate over the whole 4×4 job	0.376 GMAC/s = 0.753 GOP/s 0.492 GMAC/s reusing weights	64 MACs $\div$ 170 ns. 23.5 % of peak (30.8 % reusing), because 13 of the 17 cycles are overhead.
Latency at the estimated $F_{\max}$	$\approx$ 158 ns	17 $\times$ 9.286 ns at 107.7 MHz (see §13 for why $F_{\max}$ is not a promise)

14.4 Why 13 cycles of overhead, and how it amortises

The fixed cycles are not waste in the sense of bad design — they are the physical depth of the pipeline that *makes* the design fast (§15). But for a job with only 4 vectors they dominate. Efficiency is exactly $N / (N + 13)$ when the weights are loaded, and $N / (N + 9)$ when they are reused:

N (vectors)	Cycles (load)	Cycles (reuse)	Time (load)	MACs	Array efficiency load / reuse	Effective rate (load)
4 (one 4×4 matrix)	17	13	170 ns	64	23.5 % / 30.8 %	0.38 GMAC/s
8	21	17	210 ns	128	38.1 % / 47.1 %	0.61 GMAC/s
16	29	25	290 ns	256	55.2 % / 64.0 %	0.88 GMAC/s
64	77	73	770 ns	1 024	83.1 % / 87.7 %	1.33 GMAC/s
128	141	137	1.41 μ s	2 048	90.8 % / 93.4 %	1.45 GMAC/s
255 (max, 8-bit num_vectors)	268	264	2.68 μ s	4 080	95.1 % / 96.6 %	1.52 GMAC/s

This is the weight-stationary bargain in one table. The weights are loaded once and then reused for every activation vector, so the longer you keep streaming vectors against the same weight matrix, the closer you get to the 1.6 GMAC/s ceiling. **A single isolated 4×4 multiplication is the worst case this architecture has**; it is also, unfortunately, the case that a 4×4 demonstration measures.

WEIGHT REUSE IS AUTOMATIC — YOU GET THE 13-CYCLE PATH FOR FREE

`accelerator_system_top` keeps a one-bit `weights_dirty` flag. It is set by any host write to `weight_mem` that actually reaches the SRAM, cleared on the final WLOAD cycle, and set by reset. It drives `controller.load_weights`, so a job whose weight matrix has not changed

SKIPS WLOAD ENTIRELY AND FINISHES IN N + 9

. Nothing in the host protocol changes and the module's port list is unchanged — `fpga_top`, `vio_debug_top` and `accel_axi` all benefit without modification. Verified in `tb_accelerator_system_top` T7/T8: a reuse job measures 13 cycles with WLOAD never entered and results bit-identical to a full-load job, and rewriting one weight row puts the next job back on the 17-cycle path.

14.5 Energy per matrix multiplication

Combining §12.6's power figures with the 170 ns job time gives the metric accelerator work is usually judged by. These are derived from a **vector-less, medium-confidence** power estimate (§12.6), so treat them as the right order of magnitude rather than as measured joules:

Scope	Power	Energy per 4×4 multiply	Per MAC	Reusing weights
Whole chip, including static leakage	0.145 W	24.7 nJ	385 pJ	18.9 nJ / 295 pJ
Whole design, dynamic only	0.042 W	7.1 nJ	112 pJ	5.5 nJ / 85 pJ
The accelerator alone (<code>u_acc</code>)	0.012 W	2.0 nJ	32 pJ	1.6 nJ / 24 pJ
The 4×4 PE array alone (<code>u_fabric</code>)	0.003 W	0.51 nJ	8 pJ	0.39 nJ / 6 pJ

Two honest observations. First, the gap between the 8 pJ/MAC of the arithmetic and the 385 pJ/MAC of the whole chip is almost entirely **static leakage and debug instrumentation** — the accelerator is a 2 % tenant of a device it does not need. Second, because efficiency is only 23.5 % at $N = 4$, the energy per MAC falls sharply with larger N : at $N = 255$ the same arithmetic-only figure drops to roughly 2 pJ/MAC, since the array is busy 95 % of the time instead of 24 %.

14.6 What is *not* in the 17 cycles

Activity	Cost	Why it is excluded
Loading the activation matrix into <code>a_mem</code>	Host-paced	Done before <code>start</code> , over JTAG through the VIO. Milliseconds of human/USB latency, not clock cycles. The accelerator is idle throughout.
Loading the weight matrix into <code>weight_mem</code>	Host-paced	Same: written over the host port before <code>start</code> . The 4 WLOAD cycles are the <code>array</code> load, not the memory write.
Reading the 16 results back out	Host-paced	<code>output_mem</code> holds the results indefinitely after <code>done</code> ; the host reads them at its leisure. Over JTAG this is by far the slowest part of the whole exercise, and it is entirely an interface cost.
Power-on reset	65 536 cycles (655 μ s)	Once per configuration, not per job (§9).

THE BRING-UP CONSEQUENCE

On the board, a 4×4 multiplication finishes in

230 NANoseconds

— roughly ten thousand times faster than a human can click a VIO button. This is precisely why `done` has to be made **STICKY**

in `vio_debug_top` and why the ILA is triggered on `start` rather than polled: the entire computation is over long before any JTAG transaction can observe it in flight. The 17-cycle figure is therefore not just a performance number, it is the reason the debug architecture looks the way it does.

15. Why this design closes timing so comfortably

The positive WNS is not luck. It is a consequence of specific architectural choices, and being able to name them is far more valuable than quoting the number:

Design property	Why it helps timing
Systolic architecture: short local links only	The defining property of a systolic array. Every PE talks only to its immediate north/west neighbours through a register. There are no long global buses, no broadcast nets, and no combinational path that crosses the array. This is the single biggest reason the design is fast , and it is why systolic arrays are used for high-frequency accelerators in the first place.
Every PE output is registered	<code>a_q</code> , <code>valid_q</code> , <code>psum_q</code> and <code>weight_q</code> are all flip-flops. Combinational logic never spans two PEs — except the <code>wload</code> shift mux, which is active only during the 4-cycle weight load when nothing else is happening.
Deeply pipelined by construction	The 8-cycle store latency is not overhead invented for timing — it falls out of the dataflow (3 down the column, +c east, +(3-c) de-skew, +1 memory, +1 strobe). A design whose <i>correctness</i> model is already pipeline-shaped tends to close timing without extra effort.
Narrow datapath: int8 in, int32 accumulate	8×8 multipliers are small. A 32-bit accumulator carry chain is one CARRY4 column group. Had this been 16×16 or float, the critical path would be several times longer.
Tiny control logic	A 5-state FSM with fixed phase durations, and an AGU built from counters and a shift register. No wide comparators, no priority encoders, no arbitration in the datapath.
No back-pressure, no handshakes	There is no ready/valid negotiation anywhere. Handshake logic creates combinational paths that run <i>backwards</i> through a pipeline and is a common source of critical paths. Fixed latency removes the whole category.
Very low utilisation (2.9 % LUT, 0.7 % FF)	The placer has the entire device to spread into and the router has abundant free channels. Net delay — often more than half the path — stays small. Timing gets dramatically harder above roughly 70–80 % utilisation.
Single clock domain, synchronous reset	No CDC, no clock groups, no recovery/removal analysis on asynchronous resets, no MMCM phase relationships to get wrong.
Modest target frequency	100 MHz on a 7-series device is a relaxed target. The board oscillator sets it; the design does not need more.

16. What erodes the margin — debug cores and routing

Build B is 0.565 ns slower than Build A. Everything below contributes, and each is a lever you can pull in the opposite direction if you ever need margin back:

Cause	Effect	What to do about it
ILA capture registers and BRAM	Large	An ILA at depth 1024 with 288 probe bits needs ≈ 8 BRAM36 — more than three times the accelerator's own memory footprint. Reduce sample depth to 256 (the whole job is 17 clocks); that saves about 6 BRAM36 and the routing pressure that goes with them.
Probing wide buses	Large	Two 128-bit probes (<code>o_rdata</code> , <code>result_flat</code>) on both the VIO and the ILA is 512 probe bits of pure routing. Probe only what you need for the question you are asking.
The debug hub's JTAG shift chain	Moderate	Unavoidable if you want any debug core. It is well constrained by the IP itself.
-flatten_hierarchy none	Moderate	Required if you want hierarchical net names to survive so you can <code>MARK_DEBUG</code> them from the XDC without editing the RTL. It genuinely costs QoR because cross-boundary optimisation is disabled. Use it while debugging, drop it for a final build.
Post-synthesis vs post-route	Systematic	Build A's +1.279 ns is a post-synthesis estimate , and on 7-series those are generally optimistic because routing is not yet real. Only post-route numbers are sign-off numbers.
High utilisation	Severe, above ~75 %	Not a problem here at 2.9 %, but the mechanism is worth knowing: congestion forces the router onto longer detours, and net delay balloons.

THE HONEST CAVEAT ABOUT BUILD A

An earlier edition of the VIO/ILA guide described this design's margin as "enormous" and advised that a negative WNS should make you suspect the constraint rather than the design. That was overstated and has been corrected.

+1.279 NS ON A 10 NS PERIOD IS ~13 % — COMFORTABLE AND PASSING, BUT NOT ENORMOUS

, and it is a post-synthesis figure without any debug logic. If *your* build reports a negative WNS, check the constraint first because that is cheap to check, but do not dismiss the result.

17. How to read a Vivado timing report line by line

17.1 The commands

```
check_timing                ;# FIRST, EVERY TIME. Unconstrained clocks/endpoints, missing I/O delay
report_clocks               ;# confirm exactly the clocks you intended exist, at the right periods
report_timing_summary       ;# WNS/TNS/WHS/THS/WPWS + failing/total endpoints + unconstrained count
report_timing -delay_type max -max_paths 10 -nworst 10 -sort_by slack ;# the 10 worst setup paths
report_timing -delay_type min -max_paths 10                        ;# the 10 worst hold paths
report_utilization           ;# LUT/FF/BRAM/DSP
report_design_analysis -congestion ;# routing congestion, if timing is hard
report_methodology           ;# TIMING-* rule violations
```

17.2 Anatomy of a single path report

```
Slack (MET) :                0.714ns (required time - arrival time)
Source:                u_acc/.../psum_q_reg[17]/C
                        (rising edge-triggered cell FDRE clocked by sys_clk {rise@0.000 fall@5.000
period=10.000})
Destination:          u_acc/.../psum_q_reg[19]/D
                        (rising edge-triggered cell FDRE clocked by sys_clk ...)
Path Group:            sys_clk
Path Type:             Setup (Max at Slow Process Corner)
Requirement:           10.000ns (sys_clk rise@10.000 - sys_clk rise@0.000)
Data Path Delay:        9.0xx ns (logic 3.xxx ns (35%) route 5.xxx ns (65%))
Logic Levels:           12 (LUT2=1 LUT6=4 CARRY4=7)
Clock Path Skew:        -0.0xx ns
Clock Uncertainty:      0.035 ns
```

Field	How to use it when diagnosing
Slack (MET / VIOLATED)	The verdict for this one path. MET = positive.
Source / Destination	The startpoint and endpoint <i>by instance name</i> . This is how you find out which module is slow. If they are both inside <code>u_acc/.../pe_*</code> , your critical path is the MAC.
Requirement	The period the tool used. If this is not what you expected, your constraint is wrong — check before touching the RTL.
Data Path Delay + logic/route split	The most diagnostic line in the report. Logic-dominated (> 60 % logic) → restructure or pipeline the logic. Route-dominated (> 60 % route) → a placement/congestion problem: improve the placer directives, reduce fan-out, or reduce utilisation.
Logic Levels	How many cells deep the cloud is. In the 1–8 range at 100 MHz you are usually fine; 20+ at 100 MHz is a red flag. A long <code>CARRY4</code> chain here points at a wide adder or comparator.
Clock Path Skew	Usually tiny inside one BUFG domain. A large value suggests the clock is not on a global buffer.
Clock Uncertainty	Jitter plus tool margin. You cannot remove it; it is there to keep you honest.

18. What to do when WNS is negative — ordered checklist

Work down this list. It is deliberately ordered cheapest-and-most-likely first.

#	Step	Detail
1	Check the constraint is what you think	Run <code>report_clocks</code> . Is the period right? Did you accidentally define the clock twice (once in your XDC, once by an IP), producing overlapping definitions and nonsense analysis? Is <code>create_clock</code> on the port or wrongly on a BUFG output?
2	Run <code>check_timing</code>	Unconstrained endpoints, no_clock, missing input/output delays. Fixing these can make WNS worse — because paths that were invisible are now being checked. That is a good thing.
3	Read the worst path	<code>report_timing -delay_type max -nworst 10</code> . Identify the module by instance name. Do not optimise anything until you know what the path actually is.
4	Logic-dominated? Pipeline it.	Insert a register in the middle of the long cloud. This is the definitive fix for setup. It costs one cycle of latency — which for a fixed-latency design like this one means updating <code>RESULT_LATENCY</code> / <code>STORE_LATENCY</code> and the AGU pipe depth consistently , or the results land at the wrong addresses.
5	Route-dominated? Attack placement.	Try stronger implementation directives (<code>Explore</code> , <code>AggressiveExplore</code>), enable <code>phys_opt</code> , reduce high fan-out nets, or floorplan with a Pblock to keep a tight cluster together.
6	Use hard blocks	For this design specifically: force the multipliers into DSP48E1 slices with (<code>* use_dsp = "yes" *</code>). Hard blocks are faster and denser than fabric equivalents.
7	Reduce the debug footprint	ILA depth 1024 → 256; drop the wide 128-bit probes you are not currently reading; remove <code>-flatten_hierarchy none</code> .
8	Add legitimate timing exceptions — carefully	Only for paths that genuinely are multi-cycle or genuinely do not matter. See the warning below.
9	Lower the clock frequency	The last resort — and for <i>this</i> project a perfectly legitimate one. None of the four interface methods depends on throughput. Add an MMCM and run the fabric at 50 MHz; the period doubles and every setup path passes with enormous room. If you do, remember to update <code>C_CLK_INPUT_FREQ_HZ</code> on <code>dbg_hub</code> to match, or you will trade a timing failure for a "debug hub not detected" failure.

TIMING EXCEPTIONS ARE HOW DESIGNS "PASS" AND THEN FAIL IN THE LAB

`set_false_path` switches analysis

OFF

for a path. If the path was real, you have not fixed anything — you have hidden it. `set_multicycle_path` must constrain

BOTH

setup and hold: `-setup N` followed by `-hold N-1`. Doing only the setup half is a classic silent hold violation that reaches silicon. Use exceptions only where you can articulate, in one sentence, why the path is genuinely not a single-cycle path.

19. Hold violations — a different animal

Restating section 5 because it is the most commonly misunderstood point in STA, and a favourite interview trap:

	Setup violation (negative WNS)	Hold violation (negative WHS)
Question asked	Is the data too slow ?	Is the data too fast ?
Edges involved	Launch edge and the next capture edge	Launch edge and that same edge
Depends on clock period?	Yes — period is in the equation	No — period is absent
Fixed by slowing the clock?	Yes	Never. Broken at every frequency.
Typical fix	Pipeline, restructure logic, better placement, hard blocks	Insert delay/buffers on the short path, fix clock skew, fix the CDC that caused it
Who usually fixes it	You, the designer	The router, automatically — a leftover means something structural is wrong
Our result	WNS +0.714 ns, 0 failing	WHS +0.045 ns, 0 failing

20. Clock domain crossing and this design

CDC is where most real timing disasters live, so it is worth stating precisely why this project is mostly exempt — and where the one crossing is.

- **The accelerator itself has exactly one clock.** Every flip-flop in `accelerator_system_top` is on `sys_clk`. There is no crossing to synchronise, so no two-flop synchronisers, no gray-coded pointers, no async FIFOs.
- **The reset is synchronous and glitch-free by construction.** A 16-bit counter holds `rst_n` low for 65 536 clocks ($\approx 655\ \mu\text{s}$) after configuration, then releases it synchronously. Synchronous release means no recovery/removal timing problem.
- **The one real crossing is JTAG $\leftrightarrow$ fabric,** inside the debug hub. You do not design or constrain it — Xilinx's debug IP owns both sides. Your one obligation is to tell it the truth about the fabric clock:

```
set_property C_CLK_INPUT_FREQ_HZ 100000000 [get_debug_cores dbg_hub]
set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
```

Getting this wrong is the number-one cause of "The debug hub core was not detected".

- **The VIO's level-to-pulse conversion is not a CDC problem, it is a rate problem.** A VIO `probe_out` is already synchronous to `clk`; the issue is that a human holds it high for milliseconds while the accelerator wants a one-clock pulse. That is solved in logic by the `oneshot` module, not by a synchroniser. See [implementation_method.pdf §4](#).

21. The constraint file, annotated

```
##---- 100 MHz on-board PL clock (Bank 13, GCLK) -----
set_property -dict {PACKAGE_PIN Y9 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -name sys_clk -period 10.000 [get_ports clk]
#   ^ physical placement AND the timing budget. Verify Y9 against the
#   Diligent master XDC for YOUR board revision before you build.

##---- Build hygiene -----
set_property BITSTREAM.GENERAL.UNCONSTRAINEDPINS {Allow} [current_design]
#   PL-only design on a Zynq part: silences the harmless unconstrained-PS
#   DRC so bitstream generation completes. It does NOT affect timing.

##---- Debug hub -----
if {[llength [get_debug_cores -quiet dbg_hub]]} {
    set_property C_CLK_INPUT_FREQ_HZ    100000000 [get_debug_cores dbg_hub]
    set_property C_ENABLE_CLK_DIVIDER   false    [get_debug_cores dbg_hub]
    set_property C_USER_SCAN_CHAIN      1        [get_debug_cores dbg_hub]
    connect_debug_port dbg_hub/clk [get_nets clk]
}
#   Tells the JTAG-side hardware what the fabric clock rate is.
```

What is deliberately absent, and why:

Constraint	Why it is not here
set_input_delay / set_output_delay	The VIO/ILA build has exactly one physical pin — the clock. No data crosses the package boundary on a user pin, so there is no external interface to model. In any design that does have real I/O, omitting these is a serious error: the paths are then simply unanalysed.
set_false_path / set_multicycle_path	Nothing in this design needs an exception. Every path is a genuine single-cycle synchronous path. Adding exceptions here would only hide information.
set_clock_groups	One clock domain. Nothing to declare asynchronous.
create_generated_clock	No clock dividers, no MMCM in the PL-only build. If you add the 50 MHz MMCM suggested in §18 step 9, this becomes necessary (or is created automatically by the MMCM IP).

22. Reproducing every number in this document

22.1 Build A — out-of-context synthesis, accelerator alone

```
create_project -in_memory -part xc7z020clg484-1
foreach f [glob <RTL_DIR>/*.sv] { read_verilog -sv $f }
read_xdc clk_only.xdc ;# contains only: create_clock -period 10.000 [get_ports clk]
synth_design -top accelerator_system_top -part xc7z020clg484-1 -mode out_of_context
report_utilization
report_timing_summary
```

22.2 Build B — the full ZedBoard debug build

```
# GUI flow: add the 13 RTL files + FPGA_IMPL/rtl/vio_debug_top.sv,
#           generate vio_0 (6 probe_in / 11 probe_out) and ila_0 (10 probes),
#           add FPGA_IMPL/vio_debug.xdc, set vio_debug_top as top.
launch_runs synth_1 -jobs 4 ; wait_on_run synth_1
launch_runs impl_1 -to_step write_bitstream -jobs 4 ; wait_on_run impl_1

open_run impl_1 ;# POST-ROUTE – these are the sign-off numbers
check_timing
report_clocks
report_timing_summary -file timing_summary_impl.rpt
report_timing -delay_type max -max_paths 10 -nworst 10 -sort_by slack -file worst_setup.rpt
report_timing -delay_type min -max_paths 10 -file worst_hold.rpt
report_utilization -file utilization_impl.rpt
report_methodology -file methodology.rpt
```

WHERE TO LOOK IN THE GUI

After implementation: *Flow Navigator* → *Implementation* → *Open Implemented Design*, then *Reports* → *Timing* → *Report Timing Summary*. The

DESIGN TIMING SUMMARY

node on the left gives you exactly the Setup / Hold / Pulse Width table quoted in §12.2. The sentence "*All user specified timing constraints are met.*" appears directly underneath it.

23. WNS for preparation — interview / viva question bank

These are the questions actually asked about this topic, with answers at the level expected. The short answer is what you say first; the follow-up is what a good candidate adds.

Q1. What is WNS?

Worst Negative Slack — the smallest setup slack among all timing paths in the design. Slack is required time minus arrival time, so WNS tells you how much margin the single worst path has. Positive WNS means every setup path meets timing at the constrained frequency; negative WNS means at least one path is too slow and the design does not close. The word "negative" in the name is historical — the value is reported whether it is positive or negative.

Q2. What WNS did your design achieve, and at what frequency?

On xc7z020clg484-1 constrained at 100 MHz (create_clock -period 10.000): the accelerator alone synthesises to **WNS +1.279 ns**, and the full ZedBoard debug build with the VIO, ILA and debug hub reports **WNS +0.714 ns, TNS 0.000, 0 failing endpoints of 5 181**, with WHS +0.045 ns and WPWS +3.750 ns, and Vivado's "*All user specified timing constraints are met.*" So roughly **7 % margin** on the 10 ns period in the build that actually runs on the board, which corresponds to an estimated $F_{\max}$ of about 108 MHz.

Q3. What is the difference between WNS and TNS?

WNS is the **worst single** path's slack; TNS is the **sum** of the negative slacks of *all* failing endpoints. WNS tells you how bad the worst offender is; TNS tells you how widespread the failure is. WNS = -0.1 ns with TNS = -0.1 ns is one path to fix; WNS = -0.1 ns with TNS = -500 ns means thousands of endpoints are failing and the problem is systemic — usually over-constraining or congestion, not one bad path.

Q4. Write the setup slack equation.

Setup slack = $T_{\text{period}} - (T_{\text{co}} + T_{\text{logic}} + T_{\text{net}} + T_{\text{setup}}) + T_{\text{skew}} - T_{\text{uncertainty}}$, or equivalently required time minus arrival time, where arrival = launch clock path + T_{co} + data path, and required = capture clock path + period - T_{setup} - uncertainty.

Q5. Can you fix a hold violation by reducing the clock frequency?

No. The clock period does not appear in the hold equation — hold compares the launch and capture at the *same* edge. A hold violation exists at every frequency including DC. You fix it by adding delay on the short path, by fixing clock skew, or by fixing the structural cause (often a missing synchroniser on a clock crossing). Setup, by contrast, is fixed by slowing the clock, because the period is the budget.

Q6. What is the critical path of your design?

The multiply-accumulate inside `processing_element`: from a PE's `psum_q` register, through the combinational `wload` output mux, across the routing to the PE below, into the 32-bit signed adder whose other operand is the 8×8 signed product `weight_q * a_in`, through the `valid_in` accumulate/bypass mux, into the next `psum_q` register. Utilisation shows **0 DSPs used**, so all sixteen multipliers were mapped into LUT fabric — which is why LUT count is 1 564 and why forcing DSP48E1 inference would be my first move if I had to raise the clock.

Q7. Why does your design have positive slack — what did you do to get it?

Mostly architecture rather than tuning. A systolic array has only **short, local, register-to-register links** — every PE talks solely to its north and west neighbours through a flip-flop, so no combinational path spans the array. The datapath is narrow (int8 × int8 into int32). The control is a 5-state FSM with fixed phase lengths and there is no ready/valid back-pressure logic, so no combinational path runs backwards through the pipeline. Utilisation is 2.9 % LUT / 0.7 % FF, so routing is uncongested. And 100 MHz is a relaxed target for a 7-series part.

Q8. Your WNS dropped from +1.279 ns to +0.714 ns. Why?

Two separate reasons. First, +1.279 ns was **post-synthesis and out-of-context** with no debug logic; post-synthesis estimates on 7-series are optimistic because routing is not yet real. Second, the board build adds the VIO, the ILA and the auto-inserted debug hub — thousands of extra flip-flops and, more importantly, competition for routing resources near the logic being probed. The endpoint count rising from 1 305 to 5 181 is the visible evidence. Both builds still pass with zero failing endpoints.

Q9. Your WHS is only +0.045 ns. Is that a problem?

No. Hold slack is pass/fail, not a margin you bank. The router deliberately inserts only just enough delay to fix hold, because every hold buffer it adds eats setup margin. +0.045 ns with **0 failing endpoints and THS 0.000** is a normal, healthy post-route result. What would worry me is a *negative* WHS, because that indicates a structural problem the router could not fix.

Q10. What is WPWS, and why is yours exactly +3.750 ns in both builds?

Worst Pulse Width Slack checks that the clock waveform itself is legal — minimum high time, minimum low time, and minimum period for each cell. With a 10 ns clock at 50 % duty the high pulse is 5.000 ns, and the 7-series minimum high pulse for a BUFG-driven flip-flop is 1.250 ns, so the slack is 5.000 - 1.250 = 3.750 ns. It is a property of the **clock**, not of the logic, which is exactly why adding the entire debug infrastructure did not change it at all.

Q11. If your timing summary showed WNS positive but you still got wrong results on the board, what would you check?

First, whether the analysis was **complete**: run `check_timing` and `report_clocks` and look at the unconstrained-path count — a beautiful WNS over 12 analysed endpoints means nothing was checked. Second, whether I hid the failure with a `set_false_path` or an incomplete `set_multicycle_path` — I would read the "User Ignored Paths" group. Third, whether the problem is not timing at all: for this design the usual suspects are the **host protocol** rather than the silicon — writing weight rows in reverse order, issuing `start` while `busy` is high so it is ignored, or driving a VIO level instead of a one-clock pulse.

Q12. Is post-synthesis timing enough to sign off?

No. Post-synthesis delays are estimates with no real routing. **Sign-off is post-route** — `open_run impl_1` then `report_timing_summary`. That is the Design Timing Summary quoted for the board build here.

Q13. How would you take this design to 200 MHz?

In order: (1) force the multipliers into DSP48E1 hard blocks with `(* use_dsp = "yes" *)` — the biggest single win, since the MAC is the critical path and 0 DSPs are currently used; (2) pipeline the multiply and the add into separate stages, then update `RESULT_LATENCY`, `STORE_LATENCY`, the AGU write-pipe depth and the controller's DRAIN length **consistently**, because this design has fixed latency and no back-pressure — an inconsistent change writes results to the wrong addresses; (3) shrink or remove the debug cores; (4) use stronger implementation directives and, if congestion appears, floorplan the array. Then re-run implementation at the new period and read the new WNS, rather than trusting the $F_{\max}$ extrapolation.

Q14. What does "0 failing endpoints of 5 181" mean?

5 181 capture points — flip-flop D pins and output ports — were analysed, and none of them has negative slack. The second number matters as much as the first: it confirms the analysis actually covered the design. If it had said "0 failing endpoints of 12", I would go looking for a missing `create_clock`.

Q15. What is clock uncertainty and can you get rid of it?

A margin the tool subtracts from the required time to account for clock jitter, MMCM/PLL phase error and modelling imprecision. You cannot remove it and should not want to — it is what keeps the analysis from being optimistic. You can *tighten* it deliberately with `set_clock_uncertainty` to build in extra guard band, which makes timing harder to close on purpose.

Q16. Simulation passed all 3 680 checks. Doesn't that prove the hardware works?

It proves the **logic** is right, not that the **silicon** can run it at 100 MHz. RTL simulation is zero-delay — every gate is instantaneous — so it can never detect a setup violation. Functional correctness and timing closure are independent, and both are required. That is why the hardware bring-up replays the same golden T5 vector on the board through the VIO: a match there proves the function *and* the timing together.

Q17. How many clock cycles does one matrix multiplication take?

17 cycles for a $4 \times 4 \times 4 \times 4$ multiplication — 170 ns at 100 MHz — and 13 cycles (130 ns) when the weight matrix has not changed since the last job. Measured from the edge that samples `start`, it breaks down as `ROWS` = 4 cycles of `WLOAD` to shift the weight matrix in, `N` = 4 cycles of `COMPUTE` (one activation vector per clock), `STORE_LATENCY` = 8 cycles of `DRAIN` while the last result is still in flight, and 1 `DONE` cycle: $4+4+8+1 = 17$. In general it is **$N + 13$** , or **$N + 9$** when `WLOAD` is skipped, and because there is no back-pressure anywhere in the design those figures are exact rather than typical. Counting the `IDLE` cycle too gives the issue interval for back-to-back jobs, 18 or 14. There is deliberately no `FLUSH` phase — see Q22.

Q18. What throughput is that, and why is it so far below the peak?

A $4 \times 4 \times 4 \times 4$ multiply is 64 MACs, so 64 MACs in 17 cycles is **3.76 MAC/cycle = 0.38 GMAC/s** at 100 MHz. The **peak** is 16 MACs/cycle = **1.6 GMAC/s = 3.2 GOP/s**, because all sixteen PEs are busy every cycle of the COMPUTE phase. The gap is entirely the 13 fixed overhead cycles: efficiency is $N/(N+13)$, which is 23.5 % at $N = 4$ but 95 % at $N = 255$. That is the weight-stationary trade-off — the weights are loaded once and amortised over however many activation vectors you stream against them, so a single isolated 4×4 multiply is the architecture's worst case.

Q19. How would you make it faster without raising the clock?

Both of the obvious levers are already implemented. **Weight reuse:** 4 of the 13 fixed cycles exist only to install the weight matrix, and the weights are stationary, so a job whose weight matrix has not changed skips WLOAD and runs in 13 cycles instead of 17 — automatically, via the `weights_dirty` flag in `accelerator_system_top`.

Removing FLUSH: an earlier revision inserted 6 more cycles between WLOAD and COMPUTE to let weight residue drain out of the psum registers; that was margin rather than function and has been deleted (Q22). Together those took a 4×4 job from 17 cycles to 17, or 13 on a repeat. What remains: **increase N** so the fixed cost amortises — no RTL change at all — or raise the clock, which costs timing margin (Q13).

Q20. What does the design consume, and where does the power go?

0.145 W total on-chip, of which 0.104 W is device static leakage and only 0.042 W is dynamic; junction temperature 26.7 °C against a maximum ambient of 83.3 °C. The interesting part is the hierarchy breakdown: the ILA takes 0.019 W, the VIO 0.007 W and the debug hub 0.003 W — **69 % of all dynamic power is instrumentation** — while the accelerator itself takes 0.012 W and the entire 4×4 PE array takes **0.003 W**. At 170 ns per multiply that is about 8 pJ per MAC for the arithmetic and 385 pJ per MAC once static leakage and debug logic are counted. Worth caveating: it is a vector-less estimate at medium confidence, because under 25 % of internal nodes had specified toggle rates.

Q21. You said the timing summary is clean. What else did you check before believing it?

Four things beyond the WNS line. `report_methodology`: 16 warnings but **zero TIMING-* violations**, which is the family that catches missing clocks and unconstrained paths — that is independent evidence the analysis was complete. `report_bus_skew`: the four gray-pointer CDC buses in the debug hub pass with 0.747 ns of skew against a 10 ns requirement. `report_clock_utilization`: both clocks are on real BUFGs and the clock pin is on an MRCC site, so skew is genuinely small rather than accidentally optimistic. `report_drc`: 0 errors, 0 critical warnings. The one finding that is actually ours is `SYNTH-6` — `a_mem`'s output register was not merged into the BRAM — which costs nothing today but would become the critical path if the MAC were ever moved into DSP48E1 slices.

Q22. Your FSM has no FLUSH phase. Didn't the weight load leave residue in the psum registers?

It does, and it does not matter. Two independent reasons. **Accumulation is never at risk:** in `MAC.sv` the accumulator is `psum_q <= valid_in ? (psum_in + product) : psum_in` — a PE's own `psum_q` is only ever an *output* of the adder, never an input, so the chain is fed entirely from the row above and stale content is overwritten rather than accumulated into. **The residue can never reach memory:** it drains out of the bottom of the array at one row per cycle and clears the de-skewed result bus on cycle 12, while the first `out_we` — the AGU's write-strobe pipe, keyed off `compute_phase` and `STORE_LATENCY` deep — does not fire until cycle 13. The two trains move at the same rate and cannot converge. Removing FLUSH cut 6 cycles from every job with no functional change, confirmed by 187 checks at system level including two full signed $4 \times 4 \times 4 \times 4$ multiplications. The honest caveat: **the margin is now zero**, so anyone adding a pipeline stage in the skew → array → de-skew → pack path must re-derive that table.

24. Glossary and cheat sheet

Term	One-line definition
STA	Static Timing Analysis — exhaustive, vectorless delay checking of every path.
Slack	Required time – arrival time. Positive = passes.
WNS	Worst Negative Slack — the minimum setup slack in the design. The headline number.
TNS	Total Negative Slack — sum over all failing setup endpoints. 0.000 when nothing fails.
WHS / THS	Worst / Total Hold Slack. Period-independent; cannot be fixed by slowing down.
WPWS / TPWS	Worst / Total Pulse Width Slack — is the clock waveform itself legal.
Setup time	Data must be stable this long before the capture edge.
Hold time	Data must be stable this long after the capture edge.
T_{co}	Clock-to-Q: delay from the clock edge to the flip-flop output changing.
Startpoint / Endpoint	Where a path launches / where it is captured. Endpoint counts appear in the summary.
Clock skew	Capture clock delay – launch clock delay. Helps setup, hurts hold when positive.
Clock uncertainty	Jitter + tool margin subtracted from required time. Always makes analysis stricter.
Levels of logic	Number of cells in the combinational cloud on a path.
Critical path	The path with the least slack — here, the PE multiply-accumulate.
F_{max}	$1 / (T_{period} - WNS)$. An extrapolation, not a guarantee.
Timing closure	The state where all setup, hold and pulse-width checks pass post-route.
Timing exception	false_path / multicycle_path / max_delay — tells STA the default single-cycle assumption is wrong.
Unconstrained path	A path with no clock reaching it. Not analysed at all — invisible in WNS.
CDC	Clock Domain Crossing. Not present in this accelerator; only the JTAG/debug-hub boundary, owned by Xilinx IP.
Metastability	The undefined-output condition that a setup/hold violation causes.
Bus skew	How far apart in time the bits of one bus arrive. Checked by <code>report_bus_skew</code> ; the check that matters for gray-coded CDC pointers.
Control set	The {clock, clock-enable, set/reset} triple. All 8 flops in a 7-series slice must share one; too many distinct sets means bad packing.
MRCC / SRCC	Multi- / Single-Region Clock Capable I/O pin. A clock must enter on one to reach a BUFG cleanly. <code>clk</code> is on <code>Y9 = IO_L12P_T1_MRCC_13</code> .
Static vs dynamic power	Leakage (a property of the die) vs switching (a property of your design). Here 0.104 W vs 0.042 W.
Job latency	Cycles from <code>start</code> to <code>done</code> . Here $N + 13$ (= 17 for a 4×4), or $N + 9$ (= 13) when WLOAD is skipped.
Issue interval	Cycles before the next <code>start</code> is accepted. Here $N + 14 = 18$, or $N + 10 = 14$.
Array efficiency	Fraction of cycles the PEs do useful work: $N / (N + 13)$. 23.5 % at $N = 4$, 95 % at $N = 255$.
<code>weights_dirty</code>	Internal flag in <code>accelerator_system_top</code> that skips WLOAD when the array already holds the right weight matrix.
GMAC/s	Billion multiply-accumulates per second. Peak here = 16 PEs × 100 MHz = 1.6 GMAC/s (3.2 GOP/s).

The one-minute recap

Constraint: `create_clock -period 10.000 [get_ports clk] → 100 MHz, single domain`
Critical path: PE MAC – psum mux → 8×8 LUT multiplier → 32-bit adder → valid mux → psum_q
Accelerator alone (post-synth, OOC): WNS +1.279 WHS +0.263 WPWS +3.750 0 failing / 1 305
ZedBoard debug build (post-route): WNS +0.714 WHS +0.045 WPWS +3.750 0 failing / 5 181
TNS = THS = TPWS = 0.000 → “All user specified timing constraints are met.”
Bus skew: worst slack +9.253 ns of 10.000 • **DRC:** 0 errors, 0 critical • **Methodology:** 0 TIMING-*
Margin: 7.1 % of the period • **Estimated $F_{\max}$:** ≈ 108 MHz
Resources (post-route, whole build): 4 079 LUT (7.7 %) • 6 370 FF (6.0 %) • 11 BRAM (7.9 %) • 0 DSP
Power: 0.145 W total = 0.042 dynamic + 0.104 static • PE array alone 0.003 W • T_j 26.7 °C
Cycles per 4×4 matrix multiply: **17** (= N + 13 from the start edge; 4 WLOAD + N COMPUTE + 8 DRAIN + 1 DONE • issue interval 18)
...reusing the loaded weights: **13** (= N + 9; WLOAD skipped automatically • issue interval 14)
Latency: 170 ns / 130 ns at 100 MHz • **Peak:** 1.6 GMAC/s (3.2 GOP/s) • **Average over one 4×4 job:** 0.38 GMAC/s (23.5 % of peak)
Energy: ≈ 25 nJ per multiply whole-chip, ≈ 0.5 nJ for the PE array alone
If timing ever fails: nothing here depends on throughput – drop the fabric to 50 MHz with an MMCM.

Measured data sources: post-route implementation reports in `FPGA_IMPL/reports/` (Vivado 2020.1, xc7z020clg484-1 speed grade -1, run of 4 Aug 2026) — `utilization_synth`, `control_sets_placed`, `io_placed`, `clock_utilization_routed`, `bus_skew_routed`, `power_routed`, `drc_opted`, `drc_routed`, `methodology_drc_routed`; Design Timing Summary for the debug build from `synthesis_screenshots/`; out-of-context synthesis figures from `VIO_ILA.pdf` §15.

Cycle-count derivation: phase durations read directly from `controller.sv` (`ROWS`, `STORE_LATENCY = 1+ROWS+COLS-2+1`), cross-checked against `context_mkdown/12_dataflow_and_timing.md`, and **measured in simulation** by `tb_controller` (78 checks) and `tb_accelerator_system_top` T7/T8, which count the cycles between the `start` edge and the `done` pulse rather than assuming them: N + 13 loading, N + 9 reusing.

Design reference: `context_mkdown/` — chunk 01 (processing element / MAC), chunk 02 (systolic array), chunk 07 (controller FSM), chunk 12 (dataflow and timing derivations), chunk 13 (worked example / T5 golden case).

Functional baseline: 11/11 testbenches, 3 680 checks, 0 mismatches (`verification_recheck/`).

Companion documents: `implementation_method.pdf` (methodology and the VIO/ILA flow), `VIO_ILA.pdf` (Method 2 in full), `USB_UART.pdf` (Method 3 in full).